MINI AUTONOMOUS VEHICLE · TEAM A · 9 NOVEMBER

Week 1 runs. This update walks through the code that makes it run.

Six weeks, eight tasks. Open week 1 for the architecture, the source code of every main part, the results it produced, and the next plan.

WEEK 1 · TASKS 1 AND 2

Lane following and the obstacle

Adaptive road following, obstacle detection, calibrated stop, and circumvention. Driven on track.

● DEMONSTRATED

OPEN THE WALKTHROUGH

WEEK 2 · TASKS 3, 4, 5

Signs, intersection, parking

Same pipeline as the cup: record, label from a color mask, train, decide in our code.

○ BLOCKED ON TRACK SETUP

WEEK 3 · TASK 6

One integrated program

Six states already run in one program. Integration adds the states of week 2 to the same table.

○ AFTER WEEK 2

WEEK 4 · TASK 7

Adaptive speed and cruise control

The speed rule already takes the lower of two limits. A following distance becomes the third.

○ BLOCKED ON 2ND ROBOT

WEEKS 5 AND 6 · TASK 8

The unknown course

Parking to parking on a course nobody has seen, filmed, with the archive and the slides.

○ OPEN

i Total architecture of the code — slides 3 and 4

ii The source code itself — slides 5 to 13

iii Detailed explanation of it — beside every listing

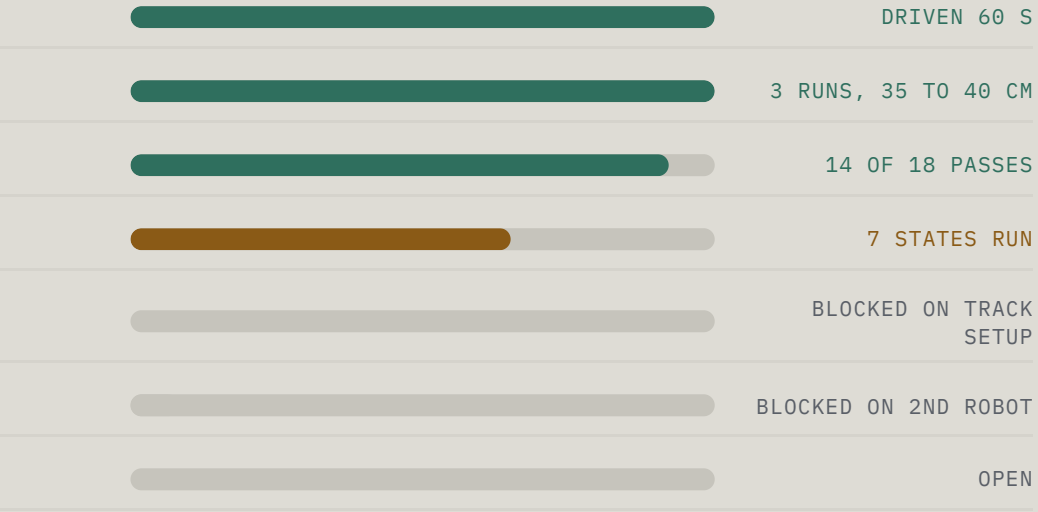
iv Results from the executed code — slides 14 to 18

v Next plan from results and bugs — slide 19

● built and tested on the track ● written, never completed a run ○ waiting on something physical enter opens · escape returns · d document · t dark

Where the project stands

W1	1 · Adaptive lane following	DRIVEN 60 S
W1	2 · Obstacle detection & stop	3 RUNS, 35 TO 40 CM
W1	2 · Obstacle circumvention maneuver	14 OF 18 PASSES
W3	6 · State machine	7 STATES RUN
W2	3, 4, 5 · Signs, intersection, parking	BLOCKED ON TRACK SETUP
W4	7 · Distance-based cruise control	BLOCKED ON 2ND ROBOT
W5	8 · Final run	OPEN



Green: verified track deployment. Amber: functional code under testing. Grey: blocked on physical track or hardware dependencies.

STATUS WHAT WE DID SINCE THE LAST DEMONSTRATION

Work completed since the last demonstration

3 NOV · TRACK, 9 MANEUVERS

5 of 9 completed

4 premature triggers; tape-angle termination restricted bypass to calibration straight.

8 NOV · TRACK, 3 OBSTACLE SITES

11 of 12 completed

Circumvention trajectory references obstacle geometry, enabling execution on curved track.

8 NOV · EMPIRICAL CALIBRATION

Metric distance model

cup_learn.py: automated incremental calibration (139 stops across 5 track locations; d = 405 / sqrt(A)).

9 NOV · TRACK, 18 MANEUVERS

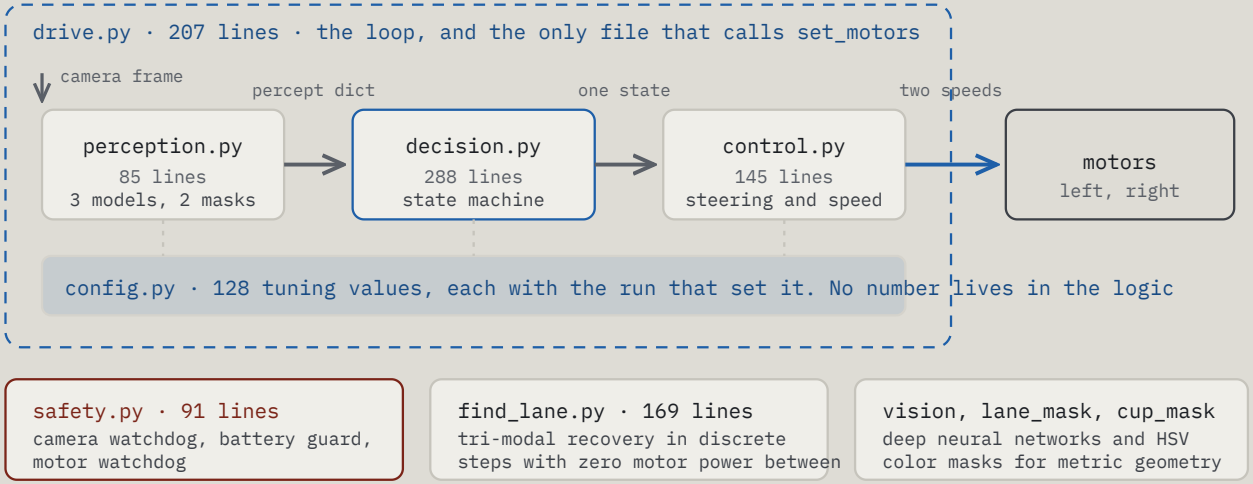
14 of 18 completed

Tri-modal recovery architecture (undo historical path, angular sweep, translational search) deployed on lost-lane events.

Documented in DECISIONS.md (OBST6 to OBST9, GAP2, LEARN1, AVOID4, LINE1). Track evaluation runs are archived on video.

I TOTAL ARCHITECTURE OF THE CODE

Code architecture: three layers and the driving loop



3136 lines in 26 Python files, all written by us. The stock NVIDIA notebooks are untouched and used as building blocks only. Data flows one way, and the layers are separated by their imports, not by a comment.

Provable in one command: grep "^import|^from" perception.py decision.py control.py. Neither decision.py nor control.py imports jetbot, so neither can reach a motor.

I TOTAL ARCHITECTURE OF THE CODE

Inference and control loop (per-frame)

1 · CAMERA

Frame

224 × 224 px, nominal 21.6 to 22.8 Hz

2 · PERCEPTION

Three models

ResNet18, TensorRT

```
lane: visible, x
curve: class, side
cup: seen, row, x
```

3 · DECISION

State machine

Six states, deterministic priority evaluation

4 · CONTROL

Steering and speed

Gain-scheduled PD controller with curve feedforward

5 · MOTORS

Wheels

Protected by battery guard, camera watchdog, and motor watchdog

The lane model runs on every frame. Curvature and obstacle models alternate across frames: evaluating all three concurrently introduced +12 ms control latency per step, inducing steering phase lag.

Measured in the Task 1 runs: 29.8 ms per step with three models, 21.6 to 22.8 steps per second. TensorRT took one lane pass from 41 ms to 10.6 ms.

II + III SOURCE CODE · DRIVE.PY

Main control loop: drive.py

drive.py is the loop. One frame in, two wheel speeds out. The only file that writes to the motors.

DRIVE.PY · RUN() · LINES 102 TO 197, SHORTENED

```
102 while time.time() - start < seconds:
106     frame = wait_for_new_frame(camera, frame)
109     percept = perception.observe(frame)
116     previous_state, state = decision.state, decision.update(percept, now)
117     if state == LINE_RECOVERY:
123         result = finder.run(history if followed else (), (-1 if turn < 0 else 1)...
128         decision.line_recovery_done(result in FOUND, last, result in KEPT_DIRECTION)
137     lane_seen = percept['lane_visible'] >= config.LANE_VISIBLE_THRESHOLD
140     if state in FOLLOWING or (state == ROAD_RECOVERY and lane_seen):
143         left, right = controller.update(percept['lane_x'], percept['curve_probs'],...
147         with watchdog.lock:
148             robot.set_motors(left, right)
149         watchdog.feed()
150     elif state in (OBSTACLE_AVOIDANCE, ROAD_RECOVERY):
189 except CameraFrozen:
191 except BatteryLow as low:
193 except KeyboardInterrupt:
195 finally:
196     watchdog.running = False
197     robot.stop()
```

106 **Never a stale image.** The camera thread dies silently on a read error and then returns the same array forever; this waits for a new frame object and times out after 0.5 s.

109 **Perception only reads.** It returns a dictionary of plain numbers and can take no action.

116 **Deterministic single state.** The main loop delegates all state transitions strictly to the decision layer.

117 **Blocking sub-routine for line recovery.** find_lane.py executes multi-second step-wise search maneuvers with zero inter-step motor torque, returning recovery status and heading retention.

143 **Control translates state into wheel speeds.** Each operational state calls its dedicated actuator mapping function.

147 **Thread-safe I2C motor bus access.** The lock prevents concurrent bus collisions between periodic control writes and asynchronous watchdog shutdowns.

189 **Exhaustive exception handling and failsafe shutdown.** All termination pathways trigger the finally block to guarantee zero-motor fail-stop.

II + III SOURCE CODE · PERCEPTION.PY

Perception layer: three models, two color masks

perception.py is the eyes. An image in, numbers out, no decisions. Models recognize, color masks measure.

PERCEPTION.PY · OBSERVE() · LINES 60 TO 83, SHORTENED

```
60 lane = self.lane_model(image)[0].float()
61 if self.curve_model is not None and self.frame_count % config.CURVE_EVERY == 0:
65 if self.obstacle_model is not None and self.frame_count % config.OBSTACLE_EVERY == 1:
71 return {
72     'lane_visible': torch.sigmoid(lane[0]).item(),
73     'lane_x': max(-1.0, min(1.0, lane[1].item())),
74     'curve_probs': curve_probs,
75     'curve_dir': curve_dir,
80     'lane_aligned': lane_mask.lane_aligned(frame),
81     'tape_angle': lane_mask.tape_angle(frame),
83     'cup_bands': [(x / (width / 2.0) - 1.0, row / float(height), pixels)
```

VISION.PY · THE FIVE MODEL CLASSES BECOME TWO SEPARATE SIGNALS

```
29 def split_curve_probs(probs):
34     straight, gentle_left, sharp_left, gentle_right, sharp_right = probs
35     return [straight, gentle_left + gentle_right, sharp_left + sharp_right],...
```

60 **The lane model runs on every frame.** Lateral feedback requires minimal latency to maintain closed-loop stability.

61 **Curvature classification runs every second frame.** Exponential probability smoothing filters single-frame classification jitter.

65 **Obstacle model alternates with curve model.** Concurrent 3-model execution added 12 ms inference delay, causing lateral oscillations after sharp turns.

71 **Pure numeric interface decoupled from PyTorch.** Eliminating tensor dependencies enables headless unit testing of the decision engine without a GPU.

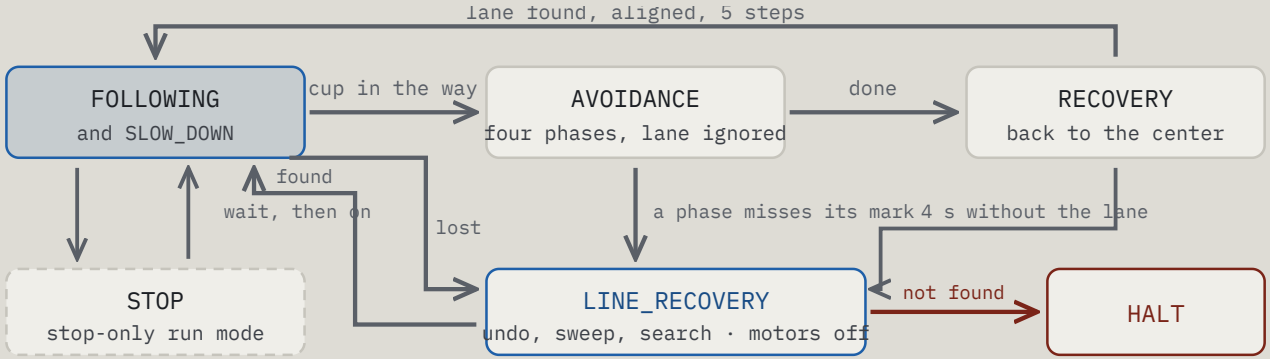
80 **Deterministic color segmentation for physical geometry:** robot heading alignment, road boundary angles, and obstacle band bounding boxes.

35 **Decoupled curvature magnitude and direction.** Five-class output separates severity (speed target) from sign (-1 to +1 for feedforward), preventing ambiguous steering inputs.

Architectural decision HYBRID (DECISIONS.md): neural network distance estimate degraded by up to 28 rows with background clutter; color segmentation remained robust.

State machine: seven states and the avoidance phases

DRIVING ACROSS THE TOP, THE WAYS OUT OF TROUBLE BELOW



PHASE 1 · ACTIVE BRAKE

0.1 s reverse pulse followed by 0.3 s dwell. Eliminates rolling inertia before initiating rotation

PHASE 2 · OUTWARD PIVOT

In-place rotation until obstacle exits visual field laterally (nominal ~90 deg yaw change)

PHASE 3 · BYPASS ARC

Tangential feed while obstacle is lateral, followed by differential arc until lane tape traverses robot underside

PHASE 4 · INWARD PIVOT

Counter-rotation terminating when lane alignment is restored and lateral offset ≤ 0.4

All phases feature sensory termination criteria bounded by safety timeouts, compensating for the lack of wheel encoders and IMU. Transition specifications: notebooks/mini_av/STATE_MACHINE.md.

II + III SOURCE CODE · DECISION.PY

State transitions and the swerve phases

decision.py is the brain, first half. It owns which state the robot is in. Here: the four phases of driving around a cup.

DECISION.PY · ENTER(), LANE_GONE() AND UPDATE_AVOIDANCE() · LINES 32 TO 235

```
 32 AVOID_PHASES = ('brake', 'turn_out', 'arc', 'turn_in')
155 def enter(self, state, now):
156     self.state, self.state_since = state, now
163 def lane_gone(self, now):
165     self.enter(LINE_RECOVERY if self.line_recovery else HALT, now)
202 elif self.avoid_phase == 'turn_out':
203     done = cup_left_view(self.turn_cup_x, band, toward) or (band is None and (
209 elif self.avoid_phase == 'arc':
210     if cup_in_the_way(band):
211         self.enter(HALT, now)
213     self.go_straight = cup_beside_curve(band, toward) or elapsed >=...
216     crossed = self.tape_under and percept['tape_row'] < config.TAPE_PAST_ROW
219     done = (crossed
225 else:
226     done = elapsed >= config.TURN_IN_MIN and lane_found(percept,...
228 if done and self.avoid_phase == AVOID_PHASES[-1]:
232     self.avoid_phase = AVOID_PHASES[AVOID_PHASES.index(self.avoid_phase) + 1]
234 elif elapsed >= limit:
235     self.lane_gone(now)
```

32 **Ordered tuple defines phase sequence.** Monotonic progression via index lookup centralizes sequence modifications.

155 **Centralized state transition via enter().** Timestamps establish deterministic baselines for timeout checks and debounce filters.

202 **Sensory-driven outward pivot termination.** Maneuver completes upon obstacle lateral exit; tape angle serves strictly as fallback.

213 **Tangential bypass before curvature onset.** Prevents premature vehicle turning into obstacle footprint; concludes upon road tape transit beneath chassis.

226 **Re-entry requires lane alignment and centering** within 0.4 of image center. Parallel heading verification prevents false locks on orthogonal tape intersections.

235 **Fault tolerance via recovery escalation.** Phase timeouts trigger LINE_RECOVERY rather than immediate non-recoverable HALT.

The four phases are the figure on the previous slide; every constant they use is in config.py.

II + III SOURCE CODE · DECISION.PY

Decision layer: priority order in decision.py

decision.py is the brain, second half. Every frame the same questions in the same order, and the first yes wins.

DECISION.PY · UPDATE() · LINES 239 TO 293, SHORTENED

```
239 if self.state in (HALT, LINE_RECOVERY):
240     return self.state
241 if self.state == OBSTACLE_AVOIDANCE:
242     self.update_avoidance(percept, now)
243     return self.state
244 if self.state == ROAD_RECOVERY:
245     self.centered_frames = self.centered_frames + 1 if lane_found(percept) else 0
246     if self.centered_frames >= config.RECOVER_FRAMES:
249         self.enter(ROAD_FOLLOWING, now)
250     elif now - self.state_since > config.RECOVER_TIMEOUT:
251         self.lane_gone(now)
254 close = cup_close(percept)
255 self.close_frames = self.close_frames + 1 if close else 0
269 elif self.close_frames >= config.OBSTACLE_FRAMES:
270     self.enter(OBSTACLE_AVOIDANCE, now)
276 if self.state == ROAD_FOLLOWING and (p_sharp > config.SLOW_DOWN_ENTER or near):
277     self.enter(SLOW_DOWN, now)
292 if now - self.lost_since >= config.LANE_LOST_TIMEOUT:
293     self.lane_gone(now)
```

239 **Deterministic priority cascade.** Sequential conditional evaluation guarantees critical safety conditions (obstacles) pre-empt lower-priority tracking (curves).

241 **Exclusive state ownership during maneuvers.** Obstacle circumvention suppresses nominal lane following until sensory or timeout completion.

244 **Conservative re-engagement qualification.** Demands 5 consecutive frames with centered position and parallel heading to preclude out-of-bounds latching.

254 **Four-frame temporal debounce.** Given alternating model execution, four frames guarantee two independent affirmative obstacle detections.

276 **Schmitt-trigger hysteresis on speed transitions.** Enters on P(sharp) >= 0.5 and exits below 0.3 to prevent limit-cycle chatter.

292 **0.5 s tracking persistence window.** Bridges brief line dropouts without state change; sustained loss initiates active recovery.

II + III SOURCE CODE · CONTROL.PY

Control layer: PD steering and adaptive speed

control.py is the hands, first half. Lane position in, a steering value and a speed out.

CONTROL.PY · UPDATE() AND WHEELS() · LINES 71 TO 144, SHORTENED

```
71  self.lane_x += config.LANE_SMOOTHING * (lane_x - self.lane_x)
74  self.d_lane_x += config.D_SMOOTHING * (self.lane_x - self.d_lane_x)
75  change = 0.0 if self.last_error is None else (self.d_lane_x - self.last_error) / dt
78  self.integral += self.lane_x * dt - self.integral * dt / config.STEERING_I_LEAK
79  integral_part = max(-config.STEERING_I_MAX, min(config.STEERING_I_MAX,...
82  schedule = config.STEERING_REF_SPEED / max(self.speed, config.STEERING_REF_SPEED)
83  wanted = (schedule * (config.STEERING_KP * self.lane_x + config.STEERING_KD * change)...
84            + self.feedforward(curve_probs, curve_dir))
85  self.steering = limit_change(wanted, self.steering, config.STEERING_RATE,...
87  target = self.target_speed(curve_probs, self.lane_x)
90  self.speed = limit_change(target, self.speed, config.SPEED_UP_RATE,...
143 left = self.speed + self.steering + config.MOTOR_TRIM
144 right = self.speed - self.steering - config.MOTOR_TRIM
```

71 **Proportional gain on instantaneous error.** Low-pass filtering lateral error introduced uncompensated phase lag and overshoot, so weight is 1.0.

74 **Derivative damping on smoothed signal.** Finite-difference differentiation of raw detections amplified frame noise into actuator jitter.

78 **Anti-windup leaky integrator.** Accumulates error in curves, decays on straights; zeroed today, as Ki = 0.25 induced instability.

82 **Velocity-dependent gain scheduling.** Control gains inversely scale with forward velocity to maintain uniform closed-loop pole placement.

83 **Model-based curvature feedforward.** Supplies a steady steering bias without a large tracking error; its direction comes from the classifier.

85 **Actuator slew-rate limiter.** Caps the steering gradient at 2.0/s, so a single-frame outlier cannot jerk the wheels.

87 **Multi-constraint velocity arbitration.** Speed is the infimum of the curvature and lateral-error limits, extensible to headway control.

143 **Empirical actuator trim compensation.** Fixed offset (+0.01 left) cancels intrinsic chassis and gearbox asymmetry.

II + III SOURCE CODE · CONTROL.PY

Control layer: what each state commands

control.py is the hands, second half. One function per state: whatever the brain picked, this is what the wheels do.

CONTROL.PY · ONE COMMAND FUNCTION PER STATE · LINES 38 TO 139, SHORTENED

```
 38 def target_speed(self, curve_probs, lane_x):
 44     model_speed = sum(p * v for p, v in zip(curve_probs, SPEEDS))
 48     return min(model_speed, steering_speed)
106 def avoid(self, phase, side, elapsed):
113     if phase == 'brake':
114         return self.stop(phase, elapsed)
115     if phase == 'pass':
116         return config.ARC_STRAIGHT, config.ARC_STRAIGHT
117     toward = -1.0 if side == 'left' else 1.0
118     if phase == 'arc':
119         outer, inner = config.ARC_WHEELS
120         return (inner, outer) if toward > 0 else (outer, inner)
121     spin = toward * turn_speed(elapsed)
122     return spin, -spin
124 def stop(self, phase, elapsed):
127     if phase == 'back':
128         return -config.BACKUP_SPEED, -config.BACKUP_SPEED
129     if phase == 'brake' and elapsed < config.BRAKE[1]:
130         return -config.BRAKE[0], -config.BRAKE[0]
133 def recover(self, side):
139     return self.wheels()
```

48 **Arbitration architecture matches Task 7.** Velocity target is the infimum of curvature-limited speed and lateral tracking speed; headway distance seamlessly integrates as a third bound.

106 **Decoupled maneuver execution.** Decision layer governs phase sequencing and termination; control layer executes open-loop wheel profiles per phase.

115 **Tangential bypass extension.** Symmetric wheel velocity past obstacle perimeter prevents early inward yaw from causing vehicle side-collisions.

119 **Empirically tuned differential arc velocities (0.36, 0.31).** Motor deadband (~0.24) necessitated empirical velocity pairs to maintain target radius.

124 **Active counter-torque braking pulse.** DC motor driver coasts at zero command; 0.1 s reverse pulse actively dissipates kinetic energy.

133 **Low-speed centering recovery.** Gentle angular velocity at minimum effective throttle smoothly re-establishes lane acquisition.

II + III SOURCE CODE · CONFIG.PY AND SAFETY.PY

Configuration constants and safety guards

config.py is the dial board, safety.py is the brake. Every tuning number in one file, away from the logic; the guards in the other.

CONFIG.PY · 5 OF 128 VALUES

```
18  STEERING_KP = 0.15      # wheel speed difference per unit of lane_x (0.35 and a cubic term made the robot swing)
37  MOTOR_TRIM = -0.01      # added to the left wheel, subtracted from the right; positive corrects a drift to the left.
63  CUP_SIZE_CM = 405.0
67  OBSTACLE_CM = 40.0      # a cup in the lane this close is the trigger. At 0.31 and braked the robot still rolls on
109 ARC_WHEELS = (0.36, 0.31)  # outer and inner wheel in the curve around the cup. A wheel moves only above about 0.24 and
```

SAFETY.PY · THE NUMBERS EVERY DRIVING PROGRAM SHARES

```
8  MOTOR_TIMEOUT = 0.25  # seconds without a motor update before the watchdog stops the motors (the loop stalled up to 1.1 s)
9  STALE_TIMEOUT = 0.5  # seconds without a new camera frame before the robot stops (the camera froze on 2026-09-20)
12 MIN_START_VOLTAGE = 11.4  # volts at rest (about 50 %) needed to start driving
```

18 **Centralized configuration repository.** All algorithmic thresholds and parameters are externalized, ensuring parameter tuning requires zero logic changes.

37 **Empirical provenance documentation.** Every constant references specific test runs, validating design choices and supporting systematic rollbacks.

63 **Calibrated metric distance estimation.** Obstacle distance follows d = 405 / sqrt(A) (cm), calibrated on 139 empirical robot stops across multiple track locations.

8 **Four-tier hardware safety watchdog:** pre-drive/under-load battery monitoring, 0.5 s camera frame heartbeat, 0.25 s motor update deadline, and unconditional failsafe stop.

Safety watchdogs are mandatory across all operational software, preventing open-loop runaway execution in the event of camera pipeline failure.

II + III SOURCE CODE · THE LEARNING PIPELINE

Model training pipeline and offline validation

This is how a model is born. The robot records itself, the masks write the labels, and no model drives before it is judged.

1 · RECORD
auto_record.py
Autonomous data acquisition via color-mask tracking; no teleoperation bias

2 · LABEL
02_auto_label.py
Automated labeling: segmentation masks generate labels.csv and contact sheets

3 · TRAIN
03_train.py
ResNet18 transfer learning per task; session-disjoint partitioning

4 · CONVERT
03_convert_trt.py
FP16 TensorRT compilation yields ~6x acceleration, fitting the 30 ms budget

5 · JUDGE
replay.py
Offline closed-loop replay benchmark evaluated per track sector and direction

03_TRAIN.PY · MAIN() · SESSION-DISJOINT TRAIN/VAL SPLIT (PREVENTS TEMPORAL DATA LEAKAGE)

```
196 val = [r for r in rows if any(v in r[1] for v in args.val)]
197 train = [r for r in rows if not any(v in r[1] for v in args.val)]
```

VISION.PY · ONE NETWORK, A NEW LAST LAYER PER TASK

```
38 def build_model(task, pretrained=False):
40     model = torchvision.models.resnet18(pretrained=pretrained)
41     model.fc = torch.nn.Linear(512, TASK_OUTPUTS[task])
```

Entire recording sessions are partitioned into validation sets, never individual frames. Contiguous frames exhibit high temporal auto-correlation; random frame-wise splitting yields artificially optimistic validation metrics due to data leakage.

Three models trained via this protocol: lane (2458 frames), curve (3276), obstacle (3147). Ground-truth labels derive from

Perception model performance and known limitations

lane

99.6 %

detection accuracy · center MAE = 0.022 · 2458 training frames

Detection rate drops to 45% when obstacle is within 18-25 cm due to visual occlusion.

curve

88.6 %

5-class accuracy · zero lateral confusion (left vs. right) · 3276 frames

6 of 37 sharp curves misclassified as gentle, causing delayed feedforward steering activation.

cup

98.1 %

detection rate up to 50 cm · 0.4 % false positive rate · 3147 training frames

71 % sensitivity at 60-73 cm; regression distance rejected in favor of optical band area.

Identified boundary limitations govern the architectural separation: semantic feature extraction is learned; safety-critical geometry is deterministically calculated.

The first two from whole laps held out of training (models/*.json). The cup model re-measured on 4629 single recorded frames, 8 November, against the cup mask.

Task 1 results: adaptive lane following, 60 s runs



Run duration benchmarked against 60 s threshold. Red indicates premature track departure. Model-based curvature feedforward successfully negotiated the sharp curve sector.

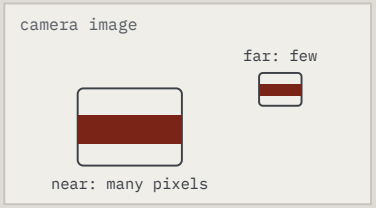
Six benchmark runs (20-21 Sep). Tracking error standard deviation: stock 0.150/0.126, baseline PD 0.087, adaptive 0.099. Benchmarking with latest codebase is pending.

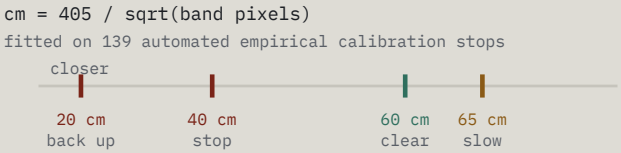
IV RESULTS · TASK 2, THE STOP

Obstacle detection and distance-gated stop

3 RUNS, 35 TO 40 CM

DISTANCE FROM THE SIZE OF THE RED BAND, NOT FROM ITS POSITION





DECISION.PY · BAND_CM() MEASURES, CUP_AHEAD() PLACES IT, CUP_CLOSE() DECIDES

```
39 return 0.0 if band[1] >= config.BAND_CUT_ROW else config.CUP_SIZE_CM / math.sqrt(band[2])
48     if abs(band[0]) < config.CUP_DEAD_AHEAD or (lane_seen and abs(band[0] -...
57 return (percept['cup_visible'] >= config.OBSTACLE_VISIBLE_THRESHOLD
58         and cup is not None and band_cm(cup) <= config.OBSTACLE_CM)
```

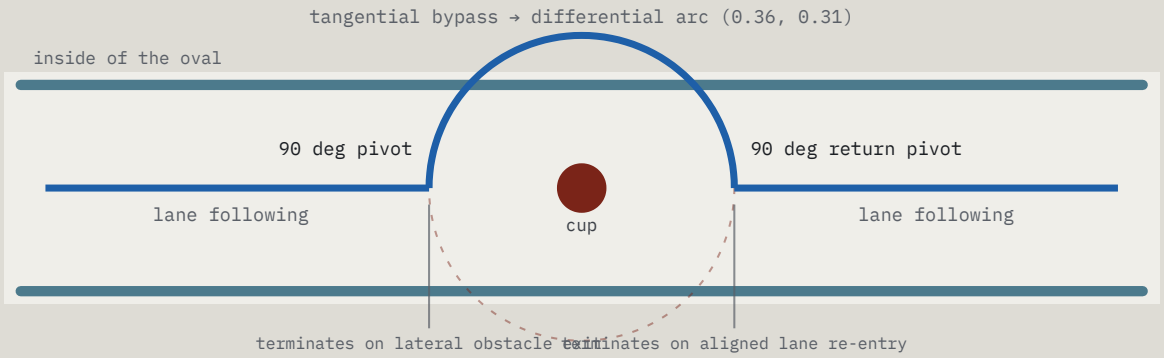
Classification model confirms obstacle presence, optical band area computes metric distance, and horizontal centroid confirms path obstruction. Path-centered obstacles trigger safety stops even during lane loss, mitigating visual occlusion.

Live evaluation runs stop_s1d2_1-3 (5 Nov) in stop-only mode: repeatable braking distances of 35-40 cm after recovery pulse. Unbraked baseline at 27 cm resulted in 10 cm vehicle stopping distance.

Obstacle circumvention and lane recovery

14 OF 18 ON 9 NOVEMBER

THE INTENDED PATH · TOP VIEW, INSIDE OF THE OVAL



Circumvention trajectory references obstacle frame rather than lane geometry: initial pivot terminates upon lateral obstacle exit, followed by tangential bypass before arc initiation. This decouples maneuver execution from track curvature.

Runs line2_1 to line2_5, 9 November: 14 of 18 completed lane re-entries (vs. 11 of 12 on 8 November). Three failures resulted from 3.9 s arc timeouts prior to visual lane re-acquisition. Obstacle contact unverified.

Root cause analysis and anomaly resolutions

WHAT WE SAW	CAUSE	FIX, AND IT HELD
Premature circumvention trigger at 50 cm distance	Obstacle model distance prediction exhibited errors up to 28 rows in presence of background pedestrian features	Metric distance calculated from optical band area (d = 405 / sqrt(A)). 11 of 12 maneuvers succeeded on 8 Nov
Circumvention maneuver failed outside calibration straight	Maneuver termination relied on absolute tape angles, which are non-stationary on curved track sections	Visual termination referenced to obstacle coordinate frame: outward pivot ends on visual exit, tangential bypass clears obstacle
Track departure resulted in terminal non-recoverable HALT state	HALT was only response to lost lane. Initial rotational scan had 0/4 recovery rate due to sub-optimal initial sweep direction	Tri-modal LINE_RECOVERY architecture (undo, sweep, search): 3/3 recovery from initial state; off-track open-floor recovery remains 0/4
Underdamped lateral oscillation (limit-cycle steering instability) at gentle curve entry (v = 0.39)	Under investigation. Error trajectory recorded: -0.11, +0.24, -0.40, +0.42 within one run	Active investigation item. Instrumented telemetry logging planned; velocity de-rating serves as interim mitigation

Each written as problem, hypothesis, change, test, result: DECISIONS.md entries OBST6, OBST9, AVOID4, LINE1 and the open item in PLAN.md S1d0.

V NEXT PLAN, FROM THE RESULTS AND THE BUGS

Next plan and pass criteria

☐	Harden off-track line recovery on open floor following aborted circumventions	pass: 3 of 4 recoveries achieved under 20 s with correct lane heading preservation
☐	Establish deterministic visual termination criterion for circumvention arc	pass: eliminate 3.9 s timeout fallback exits (accounted for 3 of 4 recorded failures)
☐	Side-view video verification of obstacle clearance envelope	pass: 8 of 10 passes filmed with zero obstacle contact; exit triggers verify 40 cm headway
☐	Isolate root cause of curve entry steering oscillation	pass: root cause identified and logged, or resolved via feedforward timing / speed envelope
☐	Team dependency: construct intersection and parking box track geometry	blocks tasks 3, 4, 5. Track geometry and tape width baseline established

Only the final item blocks subsequent milestones. Sign recognition, intersection negotiation, and parking reuse the proven perception-decision pipeline.

Steps and pass conditions from PLAN.md section 6 and the open items of 8 November. Escape returns to the roadmap, d gives the document version with the full source.